



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

Facultad de Estudios Profesionales Zona Media

PRÁCTICA 8

Contador ascendente con reinicio
usando Display de 7 segmentos

Máscaras de bits y visualización hexadecimal

Alumno: Cesar Gael Mancilla Estrada
Clave: 382031
Carrera: Ingeniería en Mecatrónica
Materia: Microcontroladores
Profesor: Ing. Jesús Padrón
Fecha: 25 de febrero de 2026

Un display de 7 segmentos convierte bits en números que todos podemos entender"

Índice

1. Introducción	2
1.1. Contexto	2
1.2. Objetivos	2
2. Marco Teórico	2
2.1. Display de 7 Segmentos	2
2.2. Tabla de segmentos	3
2.3. Funciones de máscara	3
3. Desarrollo	3
3.1. Estructura del proyecto	3
3.2. CMakeLists.txt	3
3.3. Practica_8.c	4
4. Diagramas del Circuito	5
4.1. Diagrama Esquemático	5
4.2. Diagrama en Protoboard	6
4.3. Conexiones	6
5. Preguntas de Repaso	6
6. Conclusiones	7
6.1. Aprendizajes	7
6.2. Posibles mejoras	7
7. Referencias	7

1. Introducción

1.1. Contexto

El display de 7 segmentos es uno de los dispositivos de visualización más utilizados en electrónica digital. Permite mostrar números decimales y hexadecimales mediante el encendido selectivo de siete LEDs dispuestos en forma de "8". Cada segmento se controla de forma independiente mediante un pin GPIO.

1.2. Objetivos

- Comprender la equivalencia entre valores hexadecimales y su representación en un display de 7 segmentos
- Utilizar máscaras de bits para controlar múltiples pines GPIO a la vez
- Aprender a usar las funciones `gpio_set_mask()` y `gpio_clr_mask()`
- Implementar un contador con reinicio mediante pulsador con configuración pull-up

2. Marco Teórico

2.1. Display de 7 Segmentos

Un display de 7 segmentos está compuesto por 7 LEDs etiquetados de 'a' a 'g'. Existen dos tipos:

- **Cátodo común:** Los cátodos conectados a GND. Se encienden con nivel alto (3.3V).
- **Ánodo común:** Los ánodos conectados a VCC. Se encienden con nivel bajo (0V).

Esta práctica utiliza un display de **cátodo común**.

2.2. Tabla de segmentos

Tabla 1: Valores hexadecimales para dígitos 0-F

Dígito	Hex	Segmentos activos
0	0x3F	a,b,c,d,e,f
1	0x06	b,c
2	0x5B	a,b,d,e,g
3	0x4F	a,b,c,d,g
4	0x66	b,c,f,g
5	0x6D	a,c,d,f,g
6	0x7D	a,c,d,e,f,g
7	0x07	a,b,c
8	0x7F	a,b,c,d,e,f,g
9	0x6F	a,b,c,d,f,g
A	0x77	a,b,c,e,f,g
B	0x7C	c,d,e,f,g
C	0x39	a,d,e,f
D	0x5E	a,c,d,e,g
E	0x79	a,d,e,f,g
F	0x71	a,e,f,g

2.3. Funciones de máscara

- `gpio_set_mask(mask)`: Pone en alto los pines donde la máscara tiene 1
- `gpio_clr_mask(mask)`: Pone en bajo los pines donde la máscara tiene 1

3. Desarrollo

3.1. Estructura del proyecto

- `CMakeLists.txt` - Configuración de compilación
- `Practica_8.c` - Código fuente principal
- `pico_sdk_import.cmake` - Importación del SDK

3.2. CMakeLists.txt

Listing 1: Archivo CMakeLists.txt

```

1 cmake_minimum_required(VERSION 3.13)
2
3 include(pico_sdk_import.cmake)
4 set(PICO_BOARD pico_w)
5
6 project(Practica_8)
7
8 pico_sdk_init()
9

```

```
10 add_executable(Practica_8
11     Practica_8.c
12 )
13
14 target_link_libraries(Practica_8
15     pico_stdlib
16     hardware_gpio
17 )
18
19 pico_enable_stdio_usb(Practica_8 1)
20 pico_enable_stdio_uart(Practica_8 0)
21
22 pico_add_extra_outputs(Practica_8)
```

3.3. Practica_8.c

Listing 2: Código principal del contador

```
1 /*
2  * Practica 8: Contador ascendente 0-F con reinicio
3  * Display de 7 segmentos - Uso de mascaras de bits
4  */
5
6 #include <stdio.h>
7 #include "pico/stdlib.h"
8 #include "hardware/gpio.h"
9
10 #define FIRST_GPIO 2      // Primer pin GPIO2 (segmento A)
11 #define BUTTON_GPIO 9     // Boton de reinicio GPIO9
12
13 // Tabla de segmentos para numeros 0-F
14 int hexDigits[16] = {
15     0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07,
16     0x7F, 0x6F, 0x77, 0x7C, 0x39, 0x5E, 0x79, 0x71
17 };
18
19 int main() {
20     stdio_init_all();
21     printf("Practica 8: Contador 0-F\n");
22
23     // Inicializar pines del display
24     for (int i = FIRST_GPIO; i < FIRST_GPIO + 7; i++) {
25         gpio_init(i);
26         gpio_set_dir(i, GPIO_OUT);
27     }
28
29     // Configurar boton con pull-up interno
30     gpio_init(BUTTON_GPIO);
31     gpio_set_dir(BUTTON_GPIO, GPIO_IN);
32     gpio_pull_up(BUTTON_GPIO);
33 }
```

```

34  int count = 0;
35  uint32_t mask_actual = 0;
36
37  while (true) {
38      // Detectar boton presionado
39      if (gpio_get(BUTTON_GPIO) == 0) {
40          count = 0;
41          printf("Reiniciado a 0\n");
42          sleep_ms(200);
43      }
44
45      // Actualizar display
46      uint32_t mask_nueva = hexDigits[count] << FIRST_GPIO;
47      uint32_t apagar = mask_actual & ~mask_nueva;
48      uint32_t encender = mask_nueva & ~mask_actual;
49
50      if (apagar) gpio_clr_mask(apagar);
51      if (encender) gpio_set_mask(encender);
52
53      mask_actual = mask_nueva;
54
55      // Mostrar valor
56      if (count < 10) printf("Valor: %d\n", count);
57      else printf("Valor: %c\n", 'A' + (count - 10));
58
59      sleep_ms(1000);
60      count = (count + 1) % 16;
61  }
62
63  return 0;
64  }

```

4. Diagramas del Circuito

4.1. Diagrama Esquemático

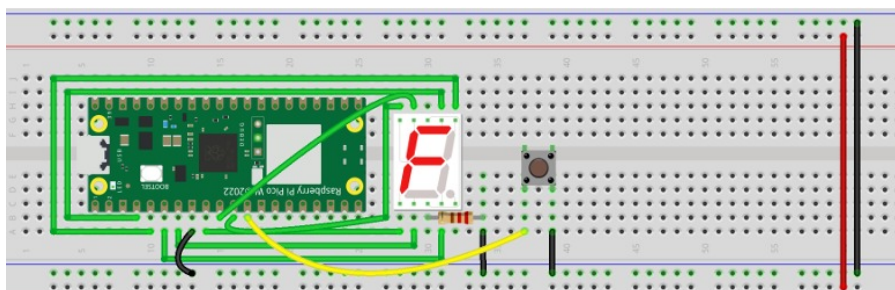


Figura 1: Diagrama esquemático del circuito

4.2. Diagrama en Protoboard

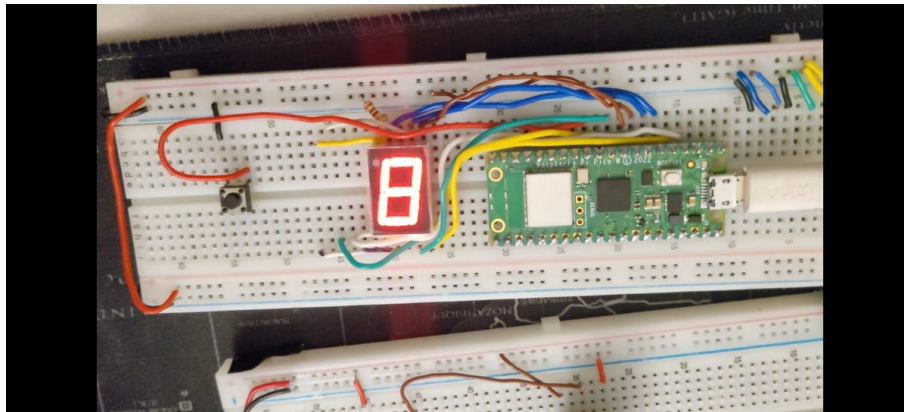


Figura 2: Conexión en protoboard

4.3. Conexiones

Tabla 2: Conexiones del display

Segmento	GPIO	Pin físico
a	GPIO2	Pin 4
b	GPIO3	Pin 5
c	GPIO4	Pin 6
d	GPIO5	Pin 7
e	GPIO6	Pin 9
f	GPIO7	Pin 10
g	GPIO8	Pin 11
Botón	GPIO9	Pin 12

5. Preguntas de Repaso

1. Equivalencia en binario de los dígitos hexadecimales

Los valores en `hexDigits` representan qué segmentos se encienden:

- `0x3F = 00111111`: enciende a,b,c,d,e,f (número 0)
- `0x06 = 00000110`: enciende b,c (número 1)
- `0x5B = 01011011`: enciende a,b,d,e,g (número 2)

Cada bit corresponde a un segmento: bit0=a, bit1=b, bit2=c, bit3=d, bit4=e, bit5=f, bit6=g.

2. Explicación de `int32_t mask = hexDigits[num] « FIRST_GPIO`

Esta línea desplaza los bits del valor hexadecimal a la posición correspondiente de los GPIO. Como `FIRST_GPIO=2`, el bit0 del hex pasa a controlar GPIO2, el bit1 a GPIO3, etc. Esto crea una máscara que puede usarse directamente con `gpio_set_mask()`.

3. Rol de la función `gpio_set_mask()`

Permite encender múltiples pines GPIO con una sola instrucción. En lugar de 7 llamadas a `gpio_put()`, se usa una máscara de 32 bits donde cada 1 indica qué pin encender. Esto hace el código más eficiente y limpio.

4. ¿Por qué no se usa resistencia externa en el botón?

Porque se utiliza la resistencia pull-up interna del RP2040 mediante `gpio_pull_up(BUTTON_GPIO)`. Con esta configuración, el pin está en nivel alto normalmente, y al presionar el botón se conecta a GND (nivel bajo). No se necesita resistencia externa.

6. Conclusiones

6.1. Aprendizajes

- Se implementó un contador hexadecimal 0-F en display de 7 segmentos
- Se utilizaron máscaras de bits para control eficiente de GPIOs
- Se comprendió la relación entre valores hex y segmentos del display
- Se aplicó pull-up interno del RP2040 para el botón

6.2. Posibles mejoras

- Agregar un segundo display para contar hasta 99
- Implementar pausa con botón adicional
- Usar interrupciones en lugar de polling
- Almacenar el valor en memoria EEPROM

7. Referencias

1. Raspberry Pi Foundation. (2024). *Raspberry Pi Pico W Datasheet*.

2. Raspberry Pi Foundation. (2024). *Pico SDK Documentation*.
3. RP2040 Datasheet. (2024). *RP2040 Datasheet*.